
MACHINE LEARNING



TOM M. MITCHELL

Machine Learning

Tom M. Mitchell

Product Details

- **Hardcover:** 432 pages ; Dimensions (in inches): 0.75 x 10.00 x 6.50
- **Publisher:** McGraw-Hill Science/Engineering/Math; (March 1, 1997)
- **ISBN:** 0070428077
- **Average Customer Review:** ★★★★★ Based on 16 reviews.
- **Amazon.com Sales Rank:** 42,816
- **Popular in:** [Redmond, WA \(#17\)](#) , [Ithaca, NY \(#9\)](#)

Editorial Reviews

From Book News, Inc. An introductory text on primary approaches to machine learning and the study of computer algorithms that improve automatically through experience. Introduce basics concepts from statistics, artificial intelligence, information theory, and other disciplines as need arises, with balanced coverage of theory and practice, and presents major algorithms with illustrations of their use. Includes chapter exercises. Online data sets and implementations of several algorithms are available on a Web site. No prior background in artificial intelligence or statistics is assumed. For advanced undergraduates and graduate students in computer science, engineering, statistics, and social sciences, as well as software professionals. *Book News, Inc.*®, Portland, OR

Book Info: Presents the key algorithms and theory that form the core of machine learning. Discusses such theoretical issues as How does learning performance vary with the number of training examples presented? and Which learning algorithms are most appropriate for various types of learning tasks? DLC: Computer algorithms.

Book Description: This book covers the field of machine learning, which is the study of algorithms that allow computer programs to automatically improve through experience. The book is intended to support upper level undergraduate and introductory level graduate courses in machine learning

PREFACE

The field of machine learning is concerned with the question of how to construct computer programs that automatically improve with experience. In recent years many successful machine learning applications have been developed, ranging from data-mining programs that learn to detect fraudulent credit card transactions, to information-filtering systems that learn users' reading preferences, to autonomous vehicles that learn to drive on public highways. At the same time, there have been important advances in the theory and algorithms that form the foundations of this field.

The goal of this textbook is to present the key algorithms and theory that form the core of machine learning. Machine learning draws on concepts and results from many fields, including statistics, artificial intelligence, philosophy, information theory, biology, cognitive science, computational complexity, and control theory. My belief is that the best way to learn about machine learning is to view it from all of these perspectives and to understand the problem settings, algorithms, and assumptions that underlie each. In the past, this has been difficult due to the absence of a broad-based single source introduction to the field. The primary goal of this book is to provide such an introduction.

Because of the interdisciplinary nature of the material, this book makes few assumptions about the background of the reader. Instead, it introduces basic concepts from statistics, artificial intelligence, information theory, and other disciplines as the need arises, focusing on just those concepts most relevant to machine learning. The book is intended for both undergraduate and graduate students in fields such as computer science, engineering, statistics, and the social sciences, and as a reference for software professionals and practitioners. Two principles that guided the writing of the book were that it should be accessible to undergraduate students and that it should contain the material I would want my own Ph.D. students to learn before beginning their doctoral research in machine learning.

A third principle that guided the writing of this book was that it should present a balance of theory and practice. Machine learning theory attempts to answer questions such as “How does learning performance vary with the number of training examples presented?” and “Which learning algorithms are most appropriate for various types of learning tasks?” This book includes discussions of these and other theoretical issues, drawing on theoretical constructs from statistics, computational complexity, and Bayesian analysis. The practice of machine learning is covered by presenting the major algorithms in the field, along with illustrative traces of their operation. Online data sets and implementations of several algorithms are available via the World Wide Web at <http://www.cs.cmu.edu/~tom/mlbook.html>. These include neural network code and data for face recognition, decision tree learning code and data for financial loan analysis, and Bayes classifier code and data for analyzing text documents. I am grateful to a number of colleagues who have helped to create these online resources, including Jason Rennie, Paul Hsiung, Jeff Shufelt, Matt Glickman, Scott Davies, Joseph O’Sullivan, Ken Lang, Andrew McCallum, and Thorsten Joachims.

ACKNOWLEDGMENTS

In writing this book, I have been fortunate to be assisted by technical experts in many of the subdisciplines that make up the field of machine learning. This book could not have been written without their help. I am deeply indebted to the following scientists who took the time to review chapter drafts and, in many cases, to tutor me and help organize chapters in their individual areas of expertise.

Avrim Blum, Jaime Carbonell, William Cohen, Greg Cooper, Mark Craven, Ken DeJong, Jerry DeJong, Tom Dietterich, Susan Epstein, Oren Etzioni, Scott Fahlman, Stephanie Forrest, David Haussler, Haym Hirsh, Rob Holte, Leslie Pack Kaelbling, Dennis Kibler, Moshe Koppel, John Koza, Miroslav Kubat, John Lafferty, Ramon Lopez de Mantaras, Sridhar Mahadevan, Stan Matwin, Andrew McCallum, Raymond Mooney, Andrew Moore, Katharina Morik, Steve Muggleton, Michael Pazzani, David Poole, Armand Prieditis, Jim Reggia, Stuart Russell, Lorenza Saitta, Claude Sammut, Jeff Schneider, Jude Shavlik, Devika Subramanian, Michael Swain, Gheorgh Tecuci, Sebastian Thrun, Peter Turney, Paul Utgoff, Manuela Veloso, Alex Waibel, Stefan Wrobel, and Yiming Yang.

I am also grateful to the many instructors and students at various universities who have field tested various drafts of this book and who have contributed their suggestions. Although there is no space to thank the hundreds of students, instructors, and others who tested earlier drafts of this book, I would like to thank the following for particularly helpful comments and discussions:

Shumeet Baluja, Andrew Banas, Andy Barto, Jim Blackson, Justin Boyan, Rich Caruana, Philip Chan, Jonathan Cheyer, Lonnie Chrisman, Dayne Freitag, Geoff Gordon, Warren Greiff, Alexander Harm, Tom Ioeberger, Thorsten

Joachim, Atsushi Kawamura, Martina Klose, Sven Koenig, Jay Modi, Andrew Ng, Joseph O'Sullivan, Patrawadee Prasangsit, Doina Precup, Bob Price, Choon Quek, Sean Slattery, Belinda Thom, Astro Teller, Will Tracz

I would like to thank Joan Mitchell for creating the index for the book. I also would like to thank Jean Harpley for help in editing many of the figures. Jane Loftus from ETP Harrison improved the presentation significantly through her copyediting of the manuscript and generally helped usher the manuscript through the intricacies of final production. Eric Munson, my editor at McGraw Hill, provided encouragement and expertise in all phases of this project.

As always, the greatest debt one owes is to one's colleagues, friends, and family. In my case, this debt is especially large. I can hardly imagine a more intellectually stimulating environment and supportive set of friends than those I have at Carnegie Mellon. Among the many here who helped, I would especially like to thank Sebastian Thrun, who throughout this project was a constant source of encouragement, technical expertise, and support of all kinds. My parents, as always, encouraged and asked "Is it done yet?" at just the right times. Finally, I must thank my family: Meghan, Shannon, and Joan. They are responsible for this book in more ways than even they know. This book is dedicated to them.

Tom M. Mitchell

CONTENTS

Preface	xv
Acknowledgments	xvi
1 Introduction	1
1.1 Well-Posed Learning Problems	2
1.2 Designing a Learning System	5
1.2.1 Choosing the Training Experience	5
1.2.2 Choosing the Target Function	7
1.2.3 Choosing a Representation for the Target Function	8
1.2.4 Choosing a Function Approximation Algorithm	9
1.2.5 The Final Design	11
1.3 Perspectives and Issues in Machine Learning	14
1.3.1 Issues in Machine Learning	15
1.4 How to Read This Book	16
1.5 Summary and Further Reading	17
Exercises	18
References	19
2 Concept Learning and the General-to-Specific Ordering	20
2.1 Introduction	20
2.2 A Concept Learning Task	21
2.2.1 Notation	22
2.2.2 The Inductive Learning Hypothesis	23
2.3 Concept Learning as Search	23
2.3.1 General-to-Specific Ordering of Hypotheses	24
2.4 FIND-S: Finding a Maximally Specific Hypothesis	26
2.5 Version Spaces and the CANDIDATE-ELIMINATION Algorithm	29
2.5.1 Representation	29
2.5.2 The LIST-THEN-ELIMINATE Algorithm	30
2.5.3 A More Compact Representation for Version Spaces	30

2.5.4	CANDIDATE-ELIMINATION Learning Algorithm	32
2.5.5	An Illustrative Example	33
2.6	Remarks on Version Spaces and CANDIDATE-ELIMINATION	37
2.6.1	Will the CANDIDATE-ELIMINATION Algorithm Converge to the Correct Hypothesis?	37
2.6.2	What Training Example Should the Learner Request Next?	37
2.6.3	How Can Partially Learned Concepts Be Used?	38
2.7	Inductive Bias	39
2.7.1	A Biased Hypothesis Space	40
2.7.2	An Unbiased Learner	40
2.7.3	The Futility of Bias-Free Learning	42
2.8	Summary and Further Reading	45
	Exercises	47
	References	50
3	Decision Tree Learning	52
3.1	Introduction	52
3.2	Decision Tree Representation	52
3.3	Appropriate Problems for Decision Tree Learning	54
3.4	The Basic Decision Tree Learning Algorithm	55
3.4.1	Which Attribute Is the Best Classifier?	55
3.4.2	An Illustrative Example	59
3.5	Hypothesis Space Search in Decision Tree Learning	60
3.6	Inductive Bias in Decision Tree Learning	63
3.6.1	Restriction Biases and Preference Biases	63
3.6.2	Why Prefer Short Hypotheses?	65
3.7	Issues in Decision Tree Learning	66
3.7.1	Avoiding Overfitting the Data	66
3.7.2	Incorporating Continuous-Valued Attributes	72
3.7.3	Alternative Measures for Selecting Attributes	73
3.7.4	Handling Training Examples with Missing Attribute Values	75
3.7.5	Handling Attributes with Differing Costs	75
3.8	Summary and Further Reading	76
	Exercises	77
	References	78
4	Artificial Neural Networks	81
4.1	Introduction	81
4.1.1	Biological Motivation	82
4.2	Neural Network Representations	82
4.3	Appropriate Problems for Neural Network Learning	83
4.4	Perceptrons	86
4.4.1	Representational Power of Perceptrons	86
4.4.2	The Perceptron Training Rule	88
4.4.3	Gradient Descent and the Delta Rule	89
4.4.4	Remarks	94

4.5	Multilayer Networks and the BACKPROPAGATION Algorithm	95
4.5.1	A Differentiable Threshold Unit	95
4.5.2	The BACKPROPAGATION Algorithm	97
4.5.3	Derivation of the BACKPROPAGATION Rule	101
4.6	Remarks on the BACKPROPAGATION Algorithm	104
4.6.1	Convergence and Local Minima	104
4.6.2	Representational Power of Feedforward Networks	105
4.6.3	Hypothesis Space Search and Inductive Bias	106
4.6.4	Hidden Layer Representations	106
4.6.5	Generalization, Overfitting, and Stopping Criterion	108
4.7	An Illustrative Example: Face Recognition	112
4.7.1	The Task	112
4.7.2	Design Choices	113
4.7.3	Learned Hidden Representations	116
4.8	Advanced Topics in Artificial Neural Networks	117
4.8.1	Alternative Error Functions	117
4.8.2	Alternative Error Minimization Procedures	119
4.8.3	Recurrent Networks	119
4.8.4	Dynamically Modifying Network Structure	121
4.9	Summary and Further Reading	122
	Exercises	124
	References	126
5	Evaluating Hypotheses	128
5.1	Motivation	128
5.2	Estimating Hypothesis Accuracy	129
5.2.1	Sample Error and True Error	130
5.2.2	Confidence Intervals for Discrete-Valued Hypotheses	131
5.3	Basics of Sampling Theory	132
5.3.1	Error Estimation and Estimating Binomial Proportions	133
5.3.2	The Binomial Distribution	135
5.3.3	Mean and Variance	136
5.3.4	Estimators, Bias, and Variance	137
5.3.5	Confidence Intervals	138
5.3.6	Two-Sided and One-Sided Bounds	141
5.4	A General Approach for Deriving Confidence Intervals	142
5.4.1	Central Limit Theorem	142
5.5	Difference in Error of Two Hypotheses	143
5.5.1	Hypothesis Testing	144
5.6	Comparing Learning Algorithms	145
5.6.1	Paired t Tests	148
5.6.2	Practical Considerations	149
5.7	Summary and Further Reading	150
	Exercises	152
	References	152
6	Bayesian Learning	154
6.1	Introduction	154
6.2	Bayes Theorem	156
6.2.1	An Example	157

6.3	Bayes Theorem and Concept Learning	158
6.3.1	Brute-Force Bayes Concept Learning	159
6.3.2	MAP Hypotheses and Consistent Learners	162
6.4	Maximum Likelihood and Least-Squared Error Hypotheses	164
6.5	Maximum Likelihood Hypotheses for Predicting Probabilities	167
6.5.1	Gradient Search to Maximize Likelihood in a Neural Net	170
6.6	Minimum Description Length Principle	171
6.7	Bayes Optimal Classifier	174
6.8	Gibbs Algorithm	176
6.9	Naive Bayes Classifier	177
6.9.1	An Illustrative Example	178
6.10	An Example: Learning to Classify Text	180
6.10.1	Experimental Results	182
6.11	Bayesian Belief Networks	184
6.11.1	Conditional Independence	185
6.11.2	Representation	186
6.11.3	Inference	187
6.11.4	Learning Bayesian Belief Networks	188
6.11.5	Gradient Ascent Training of Bayesian Networks	188
6.11.6	Learning the Structure of Bayesian Networks	190
6.12	The EM Algorithm	191
6.12.1	Estimating Means of k Gaussians	191
6.12.2	General Statement of EM Algorithm	194
6.12.3	Derivation of the k Means Algorithm	195
6.13	Summary and Further Reading	197
	Exercises	198
	References	199
7	Computational Learning Theory	201
7.1	Introduction	201
7.2	Probably Learning an Approximately Correct Hypothesis	203
7.2.1	The Problem Setting	203
7.2.2	Error of a Hypothesis	204
7.2.3	PAC Learnability	205
7.3	Sample Complexity for Finite Hypothesis Spaces	207
7.3.1	Agnostic Learning and Inconsistent Hypotheses	210
7.3.2	Conjunctions of Boolean Literals Are PAC-Learnable	211
7.3.3	PAC-Learnability of Other Concept Classes	212
7.4	Sample Complexity for Infinite Hypothesis Spaces	214
7.4.1	Shattering a Set of Instances	214
7.4.2	The Vapnik-Chervonenkis Dimension	215
7.4.3	Sample Complexity and the VC Dimension	217
7.4.4	VC Dimension for Neural Networks	218
7.5	The Mistake Bound Model of Learning	220
7.5.1	Mistake Bound for the FIND-S Algorithm	220
7.5.2	Mistake Bound for the HALVING Algorithm	221
7.5.3	Optimal Mistake Bounds	222
7.5.4	WEIGHTED-MAJORITY Algorithm	223

7.6	Summary and Further Reading	225
	Exercises	227
	References	229
8	Instance-Based Learning	230
8.1	Introduction	230
8.2	k -NEAREST NEIGHBOR LEARNING	231
8.2.1	Distance-Weighted NEAREST NEIGHBOR Algorithm	233
8.2.2	Remarks on k -NEAREST NEIGHBOR Algorithm	234
8.2.3	A Note on Terminology	236
8.3	Locally Weighted Regression	236
8.3.1	Locally Weighted Linear Regression	237
8.3.2	Remarks on Locally Weighted Regression	238
8.4	Radial Basis Functions	238
8.5	Case-Based Reasoning	240
8.6	Remarks on Lazy and Eager Learning	244
8.7	Summary and Further Reading	245
	Exercises	247
	References	247
9	Genetic Algorithms	249
9.1	Motivation	249
9.2	Genetic Algorithms	250
9.2.1	Representing Hypotheses	252
9.2.2	Genetic Operators	253
9.2.3	Fitness Function and Selection	255
9.3	An Illustrative Example	256
9.3.1	Extensions	258
9.4	Hypothesis Space Search	259
9.4.1	Population Evolution and the Schema Theorem	260
9.5	Genetic Programming	262
9.5.1	Representing Programs	262
9.5.2	Illustrative Example	263
9.5.3	Remarks on Genetic Programming	265
9.6	Models of Evolution and Learning	266
9.6.1	Lamarckian Evolution	266
9.6.2	Baldwin Effect	267
9.7	Parallelizing Genetic Algorithms	268
9.8	Summary and Further Reading	268
	Exercises	270
	References	270
10	Learning Sets of Rules	274
10.1	Introduction	274
10.2	Sequential Covering Algorithms	275
10.2.1	General to Specific Beam Search	277
10.2.2	Variations	279
10.3	Learning Rule Sets: Summary	280

10.4	Learning First-Order Rules	283
10.4.1	First-Order Horn Clauses	283
10.4.2	Terminology	284
10.5	Learning Sets of First-Order Rules: FOIL	285
10.5.1	Generating Candidate Specializations in FOIL	287
10.5.2	Guiding the Search in FOIL	288
10.5.3	Learning Recursive Rule Sets	290
10.5.4	Summary of FOIL	290
10.6	Induction as Inverted Deduction	291
10.7	Inverting Resolution	293
10.7.1	First-Order Resolution	296
10.7.2	Inverting Resolution: First-Order Case	297
10.7.3	Summary of Inverse Resolution	298
10.7.4	Generalization, θ -Subsumption, and Entailment	299
10.7.5	PROGOL	300
10.8	Summary and Further Reading	301
	Exercises	303
	References	304
11	Analytical Learning	307
11.1	Introduction	307
11.1.1	Inductive and Analytical Learning Problems	310
11.2	Learning with Perfect Domain Theories: PROLOG-EBG	312
11.2.1	An Illustrative Trace	313
11.3	Remarks on Explanation-Based Learning	319
11.3.1	Discovering New Features	320
11.3.2	Deductive Learning	321
11.3.3	Inductive Bias in Explanation-Based Learning	322
11.3.4	Knowledge Level Learning	323
11.4	Explanation-Based Learning of Search Control Knowledge	325
11.5	Summary and Further Reading	328
	Exercises	330
	References	331
12	Combining Inductive and Analytical Learning	334
12.1	Motivation	334
12.2	Inductive-Analytical Approaches to Learning	337
12.2.1	The Learning Problem	337
12.2.2	Hypothesis Space Search	339
12.3	Using Prior Knowledge to Initialize the Hypothesis	340
12.3.1	The KBANN Algorithm	340
12.3.2	An Illustrative Example	341
12.3.3	Remarks	344
12.4	Using Prior Knowledge to Alter the Search Objective	346
12.4.1	The TANGENTPROP Algorithm	347
12.4.2	An Illustrative Example	349
12.4.3	Remarks	350
12.4.4	The EBNN Algorithm	351
12.4.5	Remarks	355

12.5	Using Prior Knowledge to Augment Search	357
	Operators	357
12.5.1	The FOCL Algorithm	357
12.5.2	Remarks	360
12.6	State of the Art	361
12.7	Summary and Further Reading	362
	Exercises	363
	References	364
13	Reinforcement Learning	367
13.1	Introduction	367
13.2	The Learning Task	370
13.3	Q Learning	373
13.3.1	The Q Function	374
13.3.2	An Algorithm for Learning Q	374
13.3.3	An Illustrative Example	376
13.3.4	Convergence	377
13.3.5	Experimentation Strategies	379
13.3.6	Updating Sequence	379
13.4	Nondeterministic Rewards and Actions	381
13.5	Temporal Difference Learning	383
13.6	Generalizing from Examples	384
13.7	Relationship to Dynamic Programming	385
13.8	Summary and Further Reading	386
	Exercises	388
	References	388
Appendix	Notation	391
	Indexes	
	Author Index	394
	Subject Index	400

CHAPTER 1

INTRODUCTION

Ever since computers were invented, we have wondered whether they might be made to learn. If we could understand how to program them to learn—to improve automatically with experience—the impact would be dramatic. Imagine computers learning from medical records which treatments are most effective for new diseases, houses learning from experience to optimize energy costs based on the particular usage patterns of their occupants, or personal software assistants learning the evolving interests of their users in order to highlight especially relevant stories from the online morning newspaper. A successful understanding of how to make computers learn would open up many new uses of computers and new levels of competence and customization. And a detailed understanding of information-processing algorithms for machine learning might lead to a better understanding of human learning abilities (and disabilities) as well.

We do not yet know how to make computers learn nearly as well as people learn. However, algorithms have been invented that are effective for certain types of learning tasks, and a theoretical understanding of learning is beginning to emerge. Many practical computer programs have been developed to exhibit useful types of learning, and significant commercial applications have begun to appear. For problems such as speech recognition, algorithms based on machine learning outperform all other approaches that have been attempted to date. In the field known as data mining, machine learning algorithms are being used routinely to discover valuable knowledge from large commercial databases containing equipment maintenance records, loan applications, financial transactions, medical records, and the like. As our understanding of computers continues to mature, it

seems inevitable that machine learning will play an increasingly central role in computer science and computer technology.

A few specific achievements provide a glimpse of the state of the art: programs have been developed that successfully learn to recognize spoken words (Waibel 1989; Lee 1989), predict recovery rates of pneumonia patients (Cooper et al. 1997), detect fraudulent use of credit cards, drive autonomous vehicles on public highways (Pomerleau 1989), and play games such as backgammon at levels approaching the performance of human world champions (Tesauro 1992, 1995). Theoretical results have been developed that characterize the fundamental relationship among the number of training examples observed, the number of hypotheses under consideration, and the expected error in learned hypotheses. We are beginning to obtain initial models of human and animal learning and to understand their relationship to learning algorithms developed for computers (e.g., Laird et al. 1986; Anderson 1991; Qin et al. 1992; Chi and Bassock 1989; Ahn and Brewer 1993). In applications, algorithms, theory, and studies of biological systems, the rate of progress has increased significantly over the past decade. Several recent applications of machine learning are summarized in Table 1.1. Langley and Simon (1995) and Rumelhart et al. (1994) survey additional applications of machine learning.

This book presents the field of machine learning, describing a variety of learning paradigms, algorithms, theoretical results, and applications. Machine learning is inherently a multidisciplinary field. It draws on results from artificial intelligence, probability and statistics, computational complexity theory, control theory, information theory, philosophy, psychology, neurobiology, and other fields. Table 1.2 summarizes key ideas from each of these fields that impact the field of machine learning. While the material in this book is based on results from many diverse fields, the reader need not be an expert in any of them. Key ideas are presented from these fields using a nonspecialist's vocabulary, with unfamiliar terms and concepts introduced as the need arises.

1.1 WELL-POSED LEARNING PROBLEMS

Let us begin our study of machine learning by considering a few learning tasks. For the purposes of this book we will define learning broadly, to include any computer program that improves its performance at some task through experience. Put more precisely,

Definition: A computer program is said to **learn** from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .

For example, a computer program that learns to play checkers might improve its performance *as measured by its ability to win* at the class of tasks involving *playing checkers games*, through experience *obtained by playing games against itself*. In general, to have a well-defined learning problem, we must identify these

- Learning to recognize spoken words.

All of the most successful speech recognition systems employ machine learning in some form. For example, the SPHINX system (e.g., Lee 1989) learns speaker-specific strategies for recognizing the primitive sounds (phonemes) and words from the observed speech signal. Neural network learning methods (e.g., Waibel et al. 1989) and methods for learning hidden Markov models (e.g., Lee 1989) are effective for automatically customizing to individual speakers, vocabularies, microphone characteristics, background noise, etc. Similar techniques have potential applications in many signal-interpretation problems.

- Learning to drive an autonomous vehicle.

Machine learning methods have been used to train computer-controlled vehicles to steer correctly when driving on a variety of road types. For example, the ALVINN system (Pomerleau 1989) has used its learned strategies to drive unassisted at 70 miles per hour for 90 miles on public highways among other cars. Similar techniques have possible applications in many sensor-based control problems.

- Learning to classify new astronomical structures.

Machine learning methods have been applied to a variety of large databases to learn general regularities implicit in the data. For example, decision tree learning algorithms have been used by NASA to learn how to classify celestial objects from the second Palomar Observatory Sky Survey (Fayyad et al. 1995). This system is now used to automatically classify all objects in the Sky Survey, which consists of three terabytes of image data.

- Learning to play world-class backgammon.

The most successful computer programs for playing games such as backgammon are based on machine learning algorithms. For example, the world's top computer program for backgammon, TD-GAMMON (Tesauro 1992, 1995), learned its strategy by playing over one million practice games against itself. It now plays at a level competitive with the human world champion. Similar techniques have applications in many practical problems where very large search spaces must be examined efficiently.

TABLE 1.1

Some successful applications of machine learning.

three features: the class of tasks, the measure of performance to be improved, and the source of experience.

A checkers learning problem:

- Task T : playing checkers
- Performance measure P : percent of games won against opponents
- Training experience E : playing practice games against itself

We can specify many learning problems in this fashion, such as learning to recognize handwritten words, or learning to drive a robotic automobile autonomously.

A handwriting recognition learning problem:

- Task T : recognizing and classifying handwritten words within images
- Performance measure P : percent of words correctly classified

-
- **Artificial intelligence**
Learning symbolic representations of concepts. Machine learning as a search problem. Learning as an approach to improving problem solving. Using prior knowledge together with training data to guide learning.
 - **Bayesian methods**
Bayes' theorem as the basis for calculating probabilities of hypotheses. The naive Bayes classifier. Algorithms for estimating values of unobserved variables.
 - **Computational complexity theory**
Theoretical bounds on the inherent complexity of different learning tasks, measured in terms of the computational effort, number of training examples, number of mistakes, etc. required in order to learn.
 - **Control theory**
Procedures that learn to control processes in order to optimize predefined objectives and that learn to predict the next state of the process they are controlling.
 - **Information theory**
Measures of entropy and information content. Minimum description length approaches to learning. Optimal codes and their relationship to optimal training sequences for encoding a hypothesis.
 - **Philosophy**
Occam's razor, suggesting that the simplest hypothesis is the best. Analysis of the justification for generalizing beyond observed data.
 - **Psychology and neurobiology**
The power law of practice, which states that over a very broad range of learning problems, people's response time improves with practice according to a power law. Neurobiological studies motivating artificial neural network models of learning.
 - **Statistics**
Characterization of errors (e.g., bias and variance) that occur when estimating the accuracy of a hypothesis based on a limited sample of data. Confidence intervals, statistical tests.
-

TABLE 1.2

Some disciplines and examples of their influence on machine learning.

- Training experience E : a database of handwritten words with given classifications

A robot driving learning problem:

- Task T : driving on public four-lane highways using vision sensors
- Performance measure P : average distance traveled before an error (as judged by human overseer)
- Training experience E : a sequence of images and steering commands recorded while observing a human driver

Our definition of learning is broad enough to include most tasks that we would conventionally call "learning" tasks, as we use the word in everyday language. It is also broad enough to encompass computer programs that improve from experience in quite straightforward ways. For example, a database system

that allows users to update data entries would fit our definition of a learning system: it improves its performance at answering database queries, based on the experience gained from database updates. Rather than worry about whether this type of activity falls under the usual informal conversational meaning of the word “learning,” we will simply adopt our technical definition of the class of programs that improve through experience. Within this class we will find many types of problems that require more or less sophisticated solutions. Our concern here is not to analyze the meaning of the English word “learning” as it is used in everyday language. Instead, our goal is to define precisely a class of problems that encompasses interesting forms of learning, to explore algorithms that solve such problems, and to understand the fundamental structure of learning problems and processes.

1.2 DESIGNING A LEARNING SYSTEM

In order to illustrate some of the basic design issues and approaches to machine learning, let us consider designing a program to learn to play checkers, with the goal of entering it in the world checkers tournament. We adopt the obvious performance measure: the percent of games it wins in this world tournament.

1.2.1 Choosing the Training Experience

The first design choice we face is to choose the type of training experience from which our system will learn. The type of training experience available can have a significant impact on success or failure of the learner. One key attribute is whether the training experience provides direct or indirect feedback regarding the choices made by the performance system. For example, in learning to play checkers, the system might learn from *direct* training examples consisting of individual checkers board states and the correct move for each. Alternatively, it might have available only *indirect* information consisting of the move sequences and final outcomes of various games played. In this latter case, information about the correctness of specific moves early in the game must be inferred indirectly from the fact that the game was eventually won or lost. Here the learner faces an additional problem of *credit assignment*, or determining the degree to which each move in the sequence deserves credit or blame for the final outcome. Credit assignment can be a particularly difficult problem because the game can be lost even when early moves are optimal, if these are followed later by poor moves. Hence, learning from direct training feedback is typically easier than learning from indirect feedback.

A second important attribute of the training experience is the degree to which the learner controls the sequence of training examples. For example, the learner might rely on the teacher to select informative board states and to provide the correct move for each. Alternatively, the learner might itself propose board states that it finds particularly confusing and ask the teacher for the correct move. Or the learner may have complete control over both the board states and (indirect) training classifications, as it does when it learns by playing against itself with no teacher

present. Notice in this last case the learner may choose between experimenting with novel board states that it has not yet considered, or honing its skill by playing minor variations of lines of play it currently finds most promising. Subsequent chapters consider a number of settings for learning, including settings in which training experience is provided by a random process outside the learner's control, settings in which the learner may pose various types of queries to an expert teacher, and settings in which the learner collects training examples by autonomously exploring its environment.

A third important attribute of the training experience is how well it represents the distribution of examples over which the final system performance P must be measured. In general, learning is most reliable when the training examples follow a distribution similar to that of future test examples. In our checkers learning scenario, the performance metric P is the percent of games the system wins in the world tournament. If its training experience E consists only of games played against itself, there is an obvious danger that this training experience might not be fully representative of the distribution of situations over which it will later be tested. For example, the learner might never encounter certain crucial board states that are very likely to be played by the human checkers champion. In practice, it is often necessary to learn from a distribution of examples that is somewhat different from those on which the final system will be evaluated (e.g., the world checkers champion might not be interested in teaching the program!). Such situations are problematic because mastery of one distribution of examples will not necessarily lead to strong performance over some other distribution. We shall see that most current theory of machine learning rests on the crucial assumption that the distribution of training examples is identical to the distribution of test examples. Despite our need to make this assumption in order to obtain theoretical results, it is important to keep in mind that this assumption must often be violated in practice.

To proceed with our design, let us decide that our system will train by playing games against itself. This has the advantage that no external trainer need be present, and it therefore allows the system to generate as much training data as time permits. We now have a fully specified learning task.

A checkers learning problem:

- Task T : playing checkers
- Performance measure P : percent of games won in the world tournament
- Training experience E : games played against itself

In order to complete the design of the learning system, we must now choose

1. the exact type of knowledge to be learned
2. a representation for this target knowledge
3. a learning mechanism

1.2.2 Choosing the Target Function

The next design choice is to determine exactly what type of knowledge will be learned and how this will be used by the performance program. Let us begin with a checkers-playing program that can generate the *legal* moves from any board state. The program needs only to learn how to choose the *best* move from among these legal moves. This learning task is representative of a large class of tasks for which the legal moves that define some large search space are known a priori, but for which the best search strategy is not known. Many optimization problems fall into this class, such as the problems of scheduling and controlling manufacturing processes where the available manufacturing steps are well understood, but the best strategy for sequencing them is not.

Given this setting where we must learn to choose among the legal moves, the most obvious choice for the type of information to be learned is a program, or function, that chooses the best move for any given board state. Let us call this function *ChooseMove* and use the notation $\text{ChooseMove} : B \rightarrow M$ to indicate that this function accepts as input any board from the set of legal board states B and produces as output some move from the set of legal moves M . Throughout our discussion of machine learning we will find it useful to reduce the problem of improving performance P at task T to the problem of learning some particular *target function* such as *ChooseMove*. The choice of the target function will therefore be a key design choice.

Although *ChooseMove* is an obvious choice for the target function in our example, this function will turn out to be very difficult to learn given the kind of indirect training experience available to our system. An alternative target function—and one that will turn out to be easier to learn in this setting—is an evaluation function that assigns a numerical score to any given board state. Let us call this target function V and again use the notation $V : B \rightarrow \Re$ to denote that V maps any legal board state from the set B to some real value (we use $\Re$ to denote the set of real numbers). We intend for this target function V to assign higher scores to better board states. If the system can successfully learn such a target function V , then it can easily use it to select the best move from any current board position. This can be accomplished by generating the successor board state produced by every legal move, then using V to choose the best successor state and therefore the best legal move.

What exactly should be the value of the target function V for any given board state? Of course any evaluation function that assigns higher scores to better board states will do. Nevertheless, we will find it useful to define one particular target function V among the many that produce optimal play. As we shall see, this will make it easier to design a training algorithm. Let us therefore define the target value $V(b)$ for an arbitrary board state b in B , as follows:

1. if b is a final board state that is won, then $V(b) = 100$
2. if b is a final board state that is lost, then $V(b) = -100$
3. if b is a final board state that is drawn, then $V(b) = 0$

4. if b is not a final state in the game, then $V(b) = V(b')$, where b' is the best final board state that can be achieved starting from b and playing optimally until the end of the game (assuming the opponent plays optimally, as well).

While this recursive definition specifies a value of $V(b)$ for every board state b , this definition is not usable by our checkers player because it is not efficiently computable. Except for the trivial cases (cases 1–3) in which the game has already ended, determining the value of $V(b)$ for a particular board state requires (case 4) searching ahead for the optimal line of play, all the way to the end of the game! Because this definition is not efficiently computable by our checkers playing program, we say that it is a *nonoperational* definition. The goal of learning in this case is to discover an *operational* description of V ; that is, a description that can be used by the checkers-playing program to evaluate states and select moves within realistic time bounds.

Thus, we have reduced the learning task in this case to the problem of discovering an *operational description of the ideal target function* V . It may be very difficult in general to learn such an operational form of V perfectly. In fact, we often expect learning algorithms to acquire only some *approximation* to the target function, and for this reason the process of learning the target function is often called *function approximation*. In the current discussion we will use the symbol $\hat{V}$ to refer to the function that is actually learned by our program, to distinguish it from the ideal target function V .

1.2.3 Choosing a Representation for the Target Function

Now that we have specified the ideal target function V , we must choose a representation that the learning program will use to describe the function $\hat{V}$ that it will learn. As with earlier design choices, we again have many options. We could, for example, allow the program to represent $\hat{V}$ using a large table with a distinct entry specifying the value for each distinct board state. Or we could allow it to represent $\hat{V}$ using a collection of rules that match against features of the board state, or a quadratic polynomial function of predefined board features, or an artificial neural network. In general, this choice of representation involves a crucial tradeoff. On one hand, we wish to pick a very expressive representation to allow representing as close an approximation as possible to the ideal target function V . On the other hand, the more expressive the representation, the more training data the program will require in order to choose among the alternative hypotheses it can represent. To keep the discussion brief, let us choose a simple representation: for any given board state, the function $\hat{V}$ will be calculated as a linear combination of the following board features:

- x_1 : the number of black pieces on the board
- x_2 : the number of red pieces on the board
- x_3 : the number of black kings on the board
- x_4 : the number of red kings on the board